PART A

Experiment No.03

A.1 Aim: To Implement informed A* search methods using C,python or Java.

A.2 Prerequisite: Data Structure, Searching Techniques

A.3 Outcome:

After successful completion of this experiment students will be able to

- Ability to analyse the local and global impact of computing in searching techniques.
- Understand, identify, analyse and design the problem, implement and validate the solution for the A* search method.
- Ability to applying knowledge of computing in search technique areas.

Tools Required: C /Java

A.4 Theory:

An informed search strategy-one that uses problem-specific knowledge-can find solutions more efficiently.A key component of these algorithms is a heuristic function *h(n).*
h(n) = estimated cost of the cheapest path from node *n* to a goal node.Admissible /heuristic never over estimated i.e. h(n)≤ Actual cost.
For example, Distance between two nodes(cities)=> straight line distance and for 8-puzzel problem- Admissible heuristic can be number of misplaced tiles h(n)= 8.

A* Search technique:

It is informed search technique. It uses additional information beyond problem formulation and tree. Search is based on Evaluation function f(n). Evaluation function is based on both heuristic function h(n) and g(n).
f(n)=g(n) + h(n)
It uses two queues for its implementation: open, close Queue. Open queue is a priority queue which is arranged in ascending order of f(n)

Algorithm:
1. Create a single member queue comprising of Root node
2. If FIRST member of queue is goal then goto step 5
3. If first member of queue is not goal then remove it from queue and add to close queue.

4. Consider its children if any, and add them to queue in ascending order of evaluation function f(n).
5. If queue is not empty then goto step 2.
6. If queue is empty then goto step 6
7. Print 'success' and stop
8. Print 'failure' and stop.

Performance Comparison:

- Completeness: yes
- Optimality: yes

Time complexity :$o(b^d)$

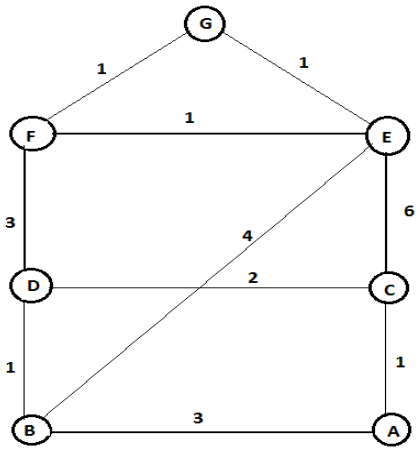
Limitation:

- It generate same node again and again
- Large Memory is required

Observation

Although A*generate many nodes it never uses those nodes for which f(n)> c* where c* is optimum cost.

Consider an example as below



OPEN/FRINGE	**CLOSE**
[A]	[ ]
[C,B]	[A]
[D,B,E,A]	[A,C]
[F,E,B,C,A]	[A,C,D]
[G,E,B,C,A,D]	[A,C,D,F]

SUCCESS

Node A:

f(B)=g(B) + h(B)=3+5=8

f(C) =g(C) + h(C)=1 + 6=7
Node C:
f(A) = g(A) + h(A)=2+7=10
f(D) = g(D) + h(D)=3+4=7
f(E) = g(E) + h(E)=7+1=8
Node D:
f(F) = g(F) + h(F)=6+1=7
f(C) = g(C) + h(C)= 5+6=11
f(B) = g(B) + h(B)=4+5=9
Node F:
f(E) = g(E) + h(E)=7+1=8
f(D) = g(D) + h(D)= 9+4=13
f(G) = g(G) + h(G)=7+0=7

Final path: A □ C □ D□F □ G

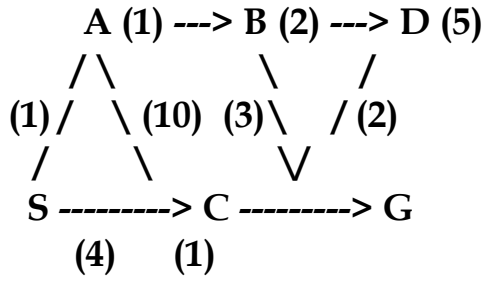
Total cost= 7

PART B

(PART B : TO BE COMPLETED BY STUDENTS)

Roll. No.: C-54	Name: Quazi Md Moinuddin
Class: TE Comps C	Batch: C3
Date of Experiment: 21.01.2025	Date of Submission: 5.05.2025
Grade:	

B.1 Software Code written by student:



```
def aStarAlgo(start_node, stop_node):
    open_set = set([start_node])  # Nodes yet to be evaluated
    closed_set = set()  # Nodes that have already been evaluated
    g = {}  # Store distance from starting node to each node
    parents = {}  # Map each node to its parent for path reconstruction

    g[start_node] = 0  # Distance from the start node to itself is zero
    parents[start_node] = start_node  # The start node is its own parent (no parent
initially)

    while len(open_set) > 0:
        n = None  # Node with the lowest f() score will be selected

        # Find the node in open_set with the lowest f(n)
        for v in open_set:
            if n is None or g[v] + heuristic(v) < g[n] + heuristic(n):
                n = v

        if n == stop_node:  # If the current node is the stop node, path is found
            path = []  # Reconstruct the path from the start node to the stop node
            total_cost = g[n]  # This stores the total cost of the path

            while parents[n] != n:  # Backtrack to the start node
                path.append(n)
                n = parents[n]
            path.append(start_node)  # Add the start node to the path
            path.reverse()  # Reverse the path to get it from start to stop

            # Print the results
            print('Path found: {}'.format(path))
            print('Total cost of the path: {}'.format(total_cost))
            return path

        # If no path is found, terminate
        if n is None:
            print('Path does not exist!')
            return None

        # Remove node n from open_set and add it to closed_set
        open_set.remove(n)
        closed_set.add(n)

        # Process each neighbor of the current node
        for (m, weight) in get_neighbors(n):
            if m in closed_set:  # If node m is already evaluated, skip it
```

```
            continue

        tentative_g = g[n] + weight  # Calculate the tentative g value (cost from start to
m)

        if m not in open_set:  # If m is not in open_set, add it
            open_set.add(m)
        elif tentative_g >= g[m]:  # If the new path is not better, skip it
            continue

        # If this path to m is better, update g and parent
        parents[m] = n
        g[m] = tentative_g

    print('Path does not exist!')  # If open_set is empty, no path was found
    return None


# Define the function to return the neighbors and their distances for a given node
def get_neighbors(v):
    if v in Graph_nodes:
        return Graph_nodes[v]  # List of neighbors and edge weights
    else:
        return []

# Define heuristic distances for each node
def heuristic(n):
    H_dist = {
        'S': 5,
        'A': 3,
        'B': 4,
        'C': 2,
        'D': 6,
        'G': 0,
    }
    return H_dist.get(n, float('inf'))  # Default to infinity if node not found in H_dist

# Define the graph structure with nodes and edges (neighbors with their respective
weights)
Graph_nodes = {
    'S': [('A', 1), ('G', 10)],  # Start node has edges to A and G with weights 1 and 10
    'A': [('B', 2), ('C', 1)],  # A has edges to B and C with weights 2 and 1
    'B': [('D', 5)],  # B has an edge to D with weight 5
    'C': [('D', 3), ('G', 4)],  # C has edges to D and G with weights 3 and 4
    'D': [('G', 2)],  # D has an edge to G with weight 2
```

```
}

# Call the A* algorithm with the start node 'S' and goal node 'G'
aStarAlgo('S', 'G')
```

B.2 Input and Output:

Input:

```
Graph_nodes={
  'S': [('A', 1), ('G', 10)],
    'A': [('B', 2), ('C', 1)],
    'B': [('D', 5)],
    'C': [('D', 3), ('G', 4)],
    'D': [('G', 2)],
}
```

Output:

```
Path found: ['S', 'A', 'C', 'G']
Total cost of the path: 6
['S', 'A', 'C', 'G']
```

B.3 Observations and learning:

Heuristic Function:

- The heuristic (h(n)) estimates the remaining cost to reach the goal, guiding the search toward the goal more efficiently.

Algorithm Logic:

- A* uses two main components:
  - g(n): Actual cost from the start node to the current node.
  - h(n): Heuristic estimate of the cost to the goal.
- The algorithm selects the node with the lowest f(n) = g(n) + h(n) for expansion.

B.4 Conclusion:

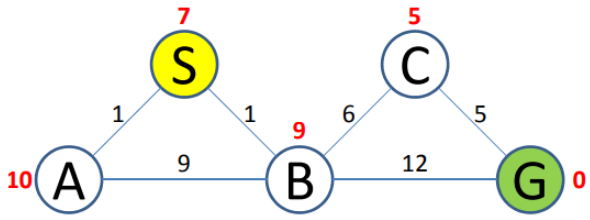
The A* search algorithm is a powerful and efficient pathfinding technique used in various fields, from AI to robotics. By combining actual costs ($g(n)$) and heuristic estimates ($h(n)$), A* prioritizes exploring nodes that are more likely to lead to the goal, ensuring that the shortest path is found with minimal computation.

Key advantages of A* include:

- **Efficiency**: It outperforms algorithms like Dijkstra's by incorporating a heuristic to guide the search.
- **Optimality**: If the heuristic is admissible (never overestimates the cost), A* guarantees the optimal path.
- **Flexibility**: A* can be adapted to different types of graphs and applications, including grid-based pathfinding, network routing, and game AI.

B.5 Question of Curiosity

Q1) Apply A* algorithm in the following example and find the cost



A* Search Process:

1. Start Node: `A`
   - g(A) = 0 (cost from start to A)
   - h(A) = 7 (heuristic to G)
   - f(A) = g(A) + h(A) = 0 + 7 = 7
2. Neighbors of A:
   - B: `g(A) + weight(A -> B) = 0 + 9 = 9`, h(B) = 5, f(B) = 9 + 5 = 14
   - S: `g(A) + weight(A -> S) = 0 + 1 = 1`, h(S) = 8, f(S) = 1 + 8 = 9
3. Select the node with the lowest f(n): `S` (f(S) = 9).
4. Expand S:
   - Neighbors of S: `A` (already visited), `B`: `g(S) + weight(S -> B) = 1 + 1 = 2`, h(B) = 5, f(B) = 2 + 5 = 7
   - Select B (f(B) = 7).
5. Expand B:
   - Neighbors of B: `C`: `g(B) + weight(B -> C) = 2 + 6 = 8`, h(C) = 3, f(C) = 8 + 3 = 11
   - G: `g(B) + weight(B -> G) = 2 + 12 = 14`, h(G) = 0, f(G) = 14 + 0 = 14
   - Select C (f(C) = 11).
6. Expand C:
   - Neighbor of C: `G`: `g(C) + weight(C -> G) = 8 + 5 = 13`, h(G) = 0, f(G) = 13 + 0 = 13
7. G is reached with f(G) = 13.

Shortest Path:

- The shortest path from A to G is: A → S → B → C → G

Total Cost:

- g(G) = 13 (Total cost from A to G).

Q2) What is the other name of informed search strategy?

a. Simple search

b. heuristic search

c. online search

d. none of the above